[ UNIVERSITY ]

[ Faculty ]

Diploma Project Brief

# ECC-Protected Sensor Data Memory on the AMD AUP ZU3

**Student:** to be assigned

[ Department, e.g. Microelectronics ]

**Supervisor:** Arman Manukyan, PhD

Yerevan, 2026

Version 0.1 · October 2026 · Draft

Topic at a Glance

# ECC-Protected Sensor Data Memory on the AMD AUP ZU3

| Work type | Degree level | Format | Duration | Open places |
|---|---|---|---|---|
| **Bachelor diploma project** | **Bachelor** | **Individual (one student)** | **2 semesters (about 30 weeks)** | **1** |

### People

**Student:** to be assigned
**[ Department, e.g. Microelectronics ]**
Supervisor: Arman Manukyan, PhD

### Timeline

Language of the work
English

### Field and requirements

| | |
|---|---|
| Research areas | Digital design · FPGA · Error-correcting codes · Functional verification · Embedded systems |
| Keywords | SystemVerilog · SECDED · Hamming code · I2C · SPI · AXI4-Lite · Zynq UltraScale+ · SVA · functional coverage |
| Tools and equipment | AMD AUP ZU3 board (Zynq UltraScale+ MPSoC), Vivado (incl. Vivado Simulator), Vitis |
| Prerequisites | Digital logic and FSMs, an HDL (Verilog/SystemVerilog), C programming, basic probability |

# Contents


1 Context and Motivation . . . 1
2 Problem and Project Idea . . . 1
3 Goals and Expected Outcomes . . . 2
4 Scope, MVP and Constraints . . . 3
5 Choosing and Emulating the Sensors . . . 4
6 ECC-Protected Memory and Register Interface . . . 8
7 Processing System and Software . . . 10
8 Verification Plan . . . 10
9 Tasks . . . 12
10 Learning Path . . . 14
11 Getting Started . . . 16
12 Work Plan and Milestones . . . 17
13 Deliverables . . . 18
14 Assessment and Acceptance Criteria . . . 19
15 Final Demonstration . . . 20
16 Extensions (Optional) . . . 20

17 Supervision, Collaboration and Risks . . . . . 21
18 Weekly Progress Tracker . . . . . 22
References and Reading List . . . . . 24
List of Abbreviations . . . . . 25
Glossary . . . . . 26

# 1. Context and Motivation

Memories in real electronic systems are not perfect. Radiation, voltage noise and ageing can flip stored bits (*soft errors*). Automotive, aerospace, medical and server systems therefore protect their memories with an Error-Correcting Code (ECC): extra check bits are stored next to the data, so that when a word is read, a single flipped bit can be **corrected** and a double flip **detected**. The same idea protects DDR memory in servers, caches in processors and on-chip memories in safety-critical Field-Programmable Gate Arrays (FPGAs) and ASICs.

At the same time, a typical embedded system collects data from **sensors** over standard serial buses such as Inter-Integrated Circuit bus ($I^2C$) and Serial Peripheral Interface (SPI), stores it and hands it to software. This project combines both topics in one small but complete system on an FPGA, designed in **SystemVerilog** and, above all, **verified** with modern verification techniques. Verification takes most of the effort in industrial chip design, so the skills practised here transfer directly to industry.

**Platform.** AMD **AUP ZU3** board with a **Zynq UltraScale+ Multiprocessor System-on-Chip (MPSoC)**: programmable logic (Programmable Logic (PL)) for the SystemVerilog design and an ARM Cortex-A53 processing system (Processing System (PS)) for the software. Tools: **Vivado** (design, simulation, synthesis, implementation) and **Vitis** (software for the PS).

**Why emulated sensors?** Real sensor chips are not required: the student writes *sensor emulators*, RTL modules that behave like real sensor chips on their bus (same registers, same protocol) and generate realistic measurement values. This makes the project independent of extra hardware, makes every scenario reproducible in simulation, and lets the student inject faults on purpose. Because the emulators follow real datasheets, real sensors can be connected later without changing the rest of the design (an extension, Chapter 16).

# 2. Problem and Project Idea

**Problem Statement**

Design, verify and demonstrate on the AUP ZU3 a SystemVerilog subsystem in which two or three emulated sensors deliver measurements over real $I^2C$ and SPI protocols, the samples are stored in an on-chip memory protected by a (39,32) SECDED Hamming code, single-bit errors are corrected and double-bit errors are detected on read, and the Zynq processing system reads the stored samples through an AXI4-Lite interface.

**Data flow.** The sensor emulators produce values (temperature, acceleration) in their internal registers. A *sensor poller* periodically reads them through an $I^2C$ master and an SPI master, and a *sample packer* turns every measurement into 32-bit words (sensor id, sequence number, timestamp, value). Each word passes the **ECC encoder**, which adds 7 check bits, and is written as a 39-bit word into a block RAM ring buffer. When the processor reads a word through the AXI4-Lite interface, the **ECC decoder** recomputes the check

bits, corrects a single-bit error, flags a double-bit error and updates error counters. A *fault-injection* unit flips chosen bits of chosen words, so that the protection can be demonstrated and verified (Figure 2.1).

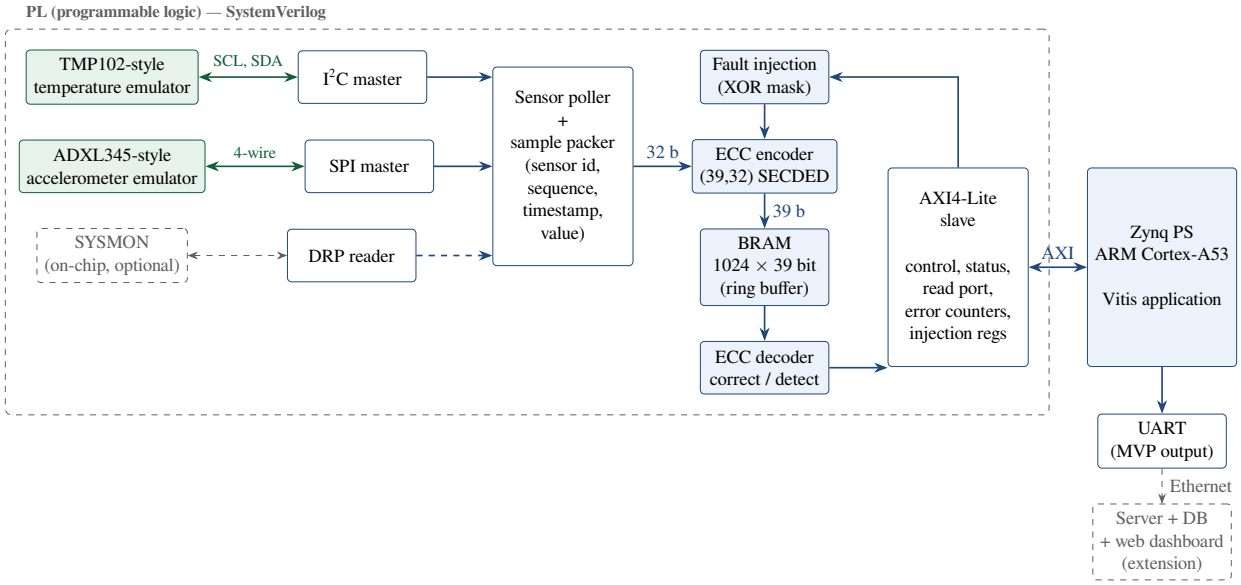



**Figure 2.1:** System architecture. Green: sensor emulators; blue: ECC memory path; dashed: optional or extension parts.

**The main challenge is verification.** Every block must be verified on its own (sensor emulators, bus masters, ECC encoder/decoder) and the complete design must be verified at top level with a self-checking, constrained-random testbench, assertions, functional coverage and fault-injection tests (Chapter 8).

**After the hardware works,** a Vitis application on the PS reads the memory and prints the decoded samples and error statistics over UART. Sending the data to a server, storing it in a database and showing it on a web page is the planned extension (Chapter 16).

# 3. Goals and Expected Outcomes

## 3.1 Project goals

1. **Select and specify the sensors:** choose the sensor chips to emulate (at least two, with different buses) and write a short specification of each emulator from the real datasheet.
2. **Design the RTL** in SystemVerilog: sensor emulators, $I^2C$ and SPI masters, sample packer, (39,32) SECDED encoder and decoder, ECC-protected BRAM ring buffer, fault injection and an AXI4-Lite register interface.
3. **Verify each module** with its own self-checking testbench (unit level), including assertions on the bus protocols.
4. **Verify the top-level design** with a class-based, constrained-random SystemVerilog environment with a scoreboard, assertions and functional coverage; reach the coverage goals of Section 8.3.
5. **Prove the ECC behaviour:** 100% of injected single-bit errors corrected and 100% of injected double-

bit errors detected, in simulation and on the board.

6. **Integrate with the Zynq PS:** synthesize and implement the design for the AUP ZU3 with no timing violations, and read the stored samples and error counters from a Vitis application.

### 3.2 Learning outcomes

On completing the project the student will be able to:

- read a sensor datasheet and turn it into an RTL specification and register map;
- design synthesizable SystemVerilog with clear interfaces, Finite-State Machines (FSMs) and parameterised modules;
- explain and implement Hamming SECDED encoding and decoding;
- build a layered, class-based SystemVerilog testbench with constrained-random stimulus, a scoreboard, SystemVerilog Assertions (SVA) and functional coverage, and run a regression;
- integrate custom RTL with the Zynq PS over AXI4-Lite and write a bare-metal Vitis application;
- close timing and analyse resource usage in Vivado.

## 4. Scope, MVP and Constraints

### 4.1 Minimum viable result (MVP)

- Two sensor emulators: a TMP102-style temperature sensor ($I^2C$) and an ADXL345-style 3-axis accelerometer (SPI), each with a configurable value generator.
- $I^2C$ master and SPI master; a sensor poller with a configurable sampling period; a sample packer.
- (39,32) SECDED encoder and decoder; BRAM ring buffer of 1024 words $\times$ 39 bits; error counters; fault injection.
- AXI4-Lite register interface (Table 6.3).
- Unit testbenches for every module and a class-based top-level verification environment with SVA and functional coverage; a regression script.
- Implementation on the AUP ZU3 without timing violations; a Vitis application that reads samples and error counters and prints them over UART, and that can trigger fault injection.

### 4.2 Out of scope (not required)

- Server, database and web dashboard. These are the main planned extension (Chapter 16), done only if the MVP is finished early.

- Real (physical) sensor chips; Linux on the PS; DMA transfers.
- A full Universal Verification Methodology (UVM) environment (optional; the required environment is class-based SystemVerilog without UVM).
- Encryption of the data: ECC protects against errors, it does not hide the data.

### 4.3 Constraints and assumptions

- One AUP ZU3 board is available to the student **[ confirm board availability and lab access ]**.
- The AUP ZU3 device (XCZU3EG) is supported by the free Vivado/Vitis edition, which includes the Vivado Simulator with SystemVerilog, SVA and functional coverage support.
- Inside the FPGA there are no real open-drain lines: the $I^2C$ SDA/SCL lines between master and emulator are modelled as *wired-AND* signals (each side drives an output-enable; the line is low if any side pulls it low).
- The proposed record format, memory size and register map in Chapter 6 are a starting point; the student may refine them with the supervisor's agreement and must document the final version.

## 5. Choosing and Emulating the Sensors

The first design task (T1.1) is to choose the sensors and to write, for each of them, an *emulator specification*: the subset of the real chip that the RTL will reproduce. This chapter gives the selection criteria, a proposed choice, the common architecture of an emulator, and the details of the two required emulators.

### 5.1 Selection criteria

A sensor is a good candidate for emulation when:

- it is a **real, widely used chip** with a public datasheet (registers, protocol, timing). The emulator then has a precise specification, and a real chip can replace it later without changing the rest of the design;
- the set of sensors uses **different buses** ($I^2C$ and SPI), so the student learns and verifies both protocols;
- the measured quantities **behave differently in time**: slow drift (temperature) and fast vibration (acceleration). This gives interesting data and test scenarios;
- the **register map is small** enough to emulate completely in a bachelor project, with no complex compensation formulas.

## 5.2 Candidate sensors

**Table 5.1:** Candidate sensors and the reason for the proposed choice.

| Chip | Measures | Bus | Assessment |
|---|---|---|---|
| **TI TMP102** [U1] | temperature | $I^2C$ | **Proposed.** 4 registers, 12-bit result, simple pointer model, very common. |
| NXP/TI LM75 | temperature | $I^2C$ | Similar to TMP102 (9–11-bit result); a valid alternative. |
| Bosch BME280 | temp., humidity, pressure | $I^2C$ / SPI | Rich, but needs calibration tables and compensation formulas. Too much for the core project. |
| **ADI ADXL345** [U2] | 3-axis acceleration | SPI / $I^2C$ | **Proposed.** Clear SPI protocol with multi-byte reads; few registers needed. |
| TDK MPU-6050 | acceleration + gyroscope | $I^2C$ | Large register map; $I^2C$ only, so it does not add SPI. |
| Microchip MCP3008 | 8-channel 10-bit ADC | SPI | Very simple SPI protocol; can model any analogue sensor (light, pressure). Good optional third sensor. |
| On-chip SYSMON [U3] | die temp., voltages | DRP | Real (not emulated) data from the FPGA itself; optional third sensor. |

The student confirms or changes this choice with the supervisor in week 5. Any change must keep one $I^2C$ sensor and one SPI sensor.

## 5.3 Common emulator architecture

All emulators share the same internal structure (Figure 5.1). Only the bus interface and the register map differ.

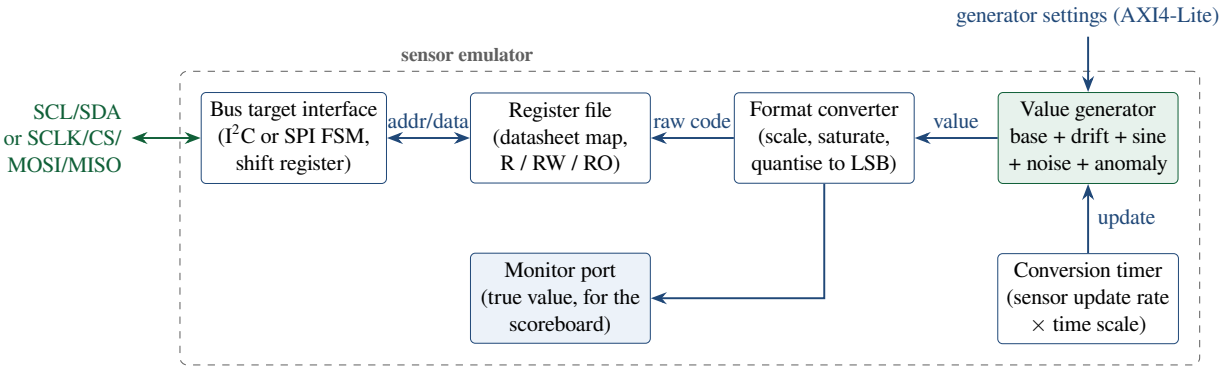



**Figure 5.1:** Common internal structure of a sensor emulator.

- **Bus target interface**: an FSM that implements the sensor's side of the protocol ($I^2C$ target or SPI target). It decodes addresses and commands and moves bytes to and from the register file.
- **Register file**: exactly the registers listed in the emulator specification, with their reset values and access types (read-only, read/write).
- **Value generator**: produces the "physical" value (Section 5.6), configured from the PS.

- **Format converter**: scales the value to the chip's data format (LSB size, two's complement, justification) and saturates it at the sensor's range.
- **Conversion timer**: updates the data registers at the sensor's conversion rate. A *time-scale* parameter makes updates faster in simulation (e.g. every few microseconds instead of every 250 ms).
- **Monitor port**: exposes the true value and an update strobe to the testbench, so the scoreboard knows exactly what was measured (Chapter 8). It is not connected in the synthesized design.

**Table 5.2:** Proposed ports of an emulator ($I^2C$ example).

| Port | Dir. | Meaning |
|---|---|---|
| `clk`, `rst_n` | in | system clock and active-low reset |
| `scl_i`, `sda_i` | in | resolved bus lines (after wired-AND) |
| `sda_oe` | out | 1 = emulator pulls SDA low (open-drain modelling) |
| `gen_cfg` | in | value-generator settings (struct, from the register interface) |
| `anomaly_trig` | in | starts an anomaly event |
| `mon_value`, `mon_update` | out | true value and update strobe (verification only) |

## 5.4 TMP102-style temperature emulator ($I^2C$)

### Protocol

The emulator is an $I^2C$ target with the 7-bit address `0x48` (binary `1001000`, the TMP102 address with its ADD0 pin tied to ground). A temperature read is a write of the pointer byte, followed by a repeated START and a two-byte read, most significant byte first (Figure 5.2).

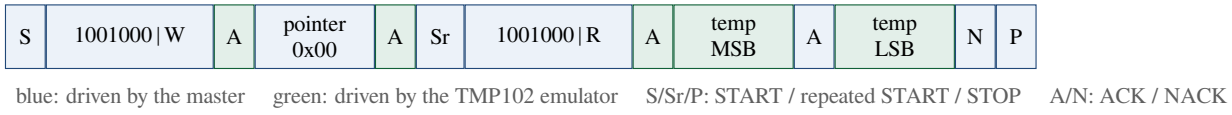



**Figure 5.2:** $I^2C$ transaction that reads the temperature register.

Required protocol features:

- START, repeated START and STOP detection; address match with ACK, and no ACK (NACK) for any other address;
- pointer-register write, followed by one- or two-byte reads and writes;
- correct behaviour at 100 kHz (standard mode) and 400 kHz (fast mode) SCL;
- SDA changed by the emulator only while SCL is low.

*Not emulated:* SMBus alert response, general call, high-speed mode, one-shot conversions.

### Registers

**Data format examples.** $+25\,°\text{C} = 400$ LSB $=$ `0x190` $\rightarrow$ register value `0x1900`; $-25\,°\text{C} = -400$ LSB $=$ `0xE70` $\rightarrow$ `0xE700`. The range is $-40$ to $+125\,°\text{C}$. The emulator must saturate at these limits. The conversion-

**Table 5.3:** TMP102 registers to emulate [U1].

| Pointer | Register | Access | Reset | Content |
|---|---|---|---|---|
| `0x00` | Temperature | R | – | 12-bit two's complement in bits [15:4], 0.0625 °C/LSB |
| `0x01` | Configuration | R/W | `0x60A0` | shutdown, thermostat mode, polarity, fault queue, resolution (read-only), conversion rate, alert |
| `0x02` | T_LOW | R/W | `0x4B00` | lower alert limit (75 °C) |
| `0x03` | T_HIGH | R/W | `0x5000` | upper alert limit (80 °C) |

rate bits (CR1:CR0) select 0.25, 1, 4 (default) or 8 conversions per second; the emulator scales them with the time-scale parameter. The alert bit (AL) should follow T_HIGH/T_LOW in comparator mode. This is a small but good verification target.

## 5.5 ADXL345-style accelerometer emulator (SPI)

### Protocol

The emulator is a 4-wire SPI target in mode 3 (clock idle high, data sampled on the rising edge), as specified for the ADXL345 [U2] (maximum 5 MHz SCLK). Every transaction starts with a command byte: bit 7 = R/W (1 = read), bit 6 = MB (1 = multi-byte, address auto-increment), bits 5:0 = register address. Reading all three axes in one burst is shown in Figure 5.3.

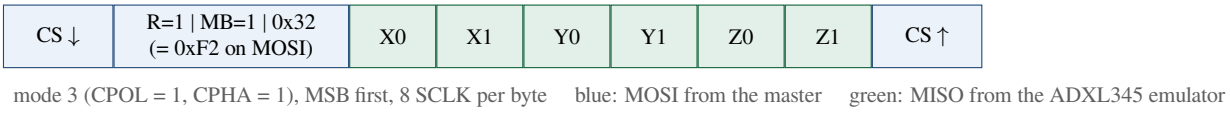



**Figure 5.3:** SPI burst read of the six data registers.

Required protocol features: single-byte read and write, multi-byte read with address auto-increment, CS framing (a transaction ends when CS goes high), and MISO driven only while CS is low. *Not emulated:* 3-wire SPI, FIFO, interrupts, tap/free-fall detection, self-test, offset registers.

### Registers

**Table 5.4:** ADXL345 registers to emulate [U2].

| Address | Register | Access | Reset | Content |
|---|---|---|---|---|
| `0x00` | DEVID | R | `0xE5` | fixed device id, for a first "is it alive?" test |
| `0x2C` | BW_RATE | R/W | `0x0A` | output data rate (default 100 Hz) |
| `0x2D` | POWER_CTL | R/W | `0x00` | bit 3 = Measure; data registers update only when set |
| `0x30` | INT_SOURCE | R | – | bit 7 = DATA_READY (optional) |
| `0x31` | DATA_FORMAT | R/W | `0x00` | range ($\pm 2/4/8/16\,g$), FULL_RES, justify |
| `0x32–0x37` | DATAX0 …DATAZ1 | R | – | X, Y, Z as 16-bit two's complement, low byte first |

**Data format.** In full-resolution mode, the scale is 3.9 m$g$/LSB (about 256 LSB per $g$ in the $\pm 2\,g$ range). At rest the emulator reports X $\approx 0$, Y $\approx 0$, Z $\approx +256$ LSB (1 $g$). Values must saturate at the selected range.

### 5.6 Value generators

Each emulator computes its physical value $v$ at every conversion step $n$ as

$$v[n] = \underbrace{v_0 + k\,n}_{\text{base + drift}} + \underbrace{A\,\sin(2\pi f\,n\,T)}_{\text{periodic part}} + \underbrace{\eta[n]}_{\text{noise (LFSR)}} + \underbrace{a[n]}_{\text{anomaly}}$$

and then converts it to the chip's format (scale, saturate, round to one LSB).

**Table 5.5:** Generator settings (written by the PS through the GEN_* registers).

| Setting | Temperature emulator | Accelerometer emulator |
|---|---|---|
| base $v_0$, drift $k$ | e.g. 25 °C, ± slow ramp | Z = 1 $g$, X = Y = 0 |
| periodic $A$, $f$ | optional slow oscillation | vibration on X/Y (per-axis amplitude, frequency) |
| noise level | small (a few LSB) | larger (tens of LSB) |
| anomaly | "overheating": step of $+\Delta T$ for a set time | "shock": short spike on one axis |

Implementation hints: a quarter-wave sine look-up table (e.g. 64 entries) with a phase accumulator is enough for the periodic part. The noise comes from a 16-bit maximal-length Linear-Feedback Shift Register (LFSR) scaled by the noise level. All arithmetic is fixed-point. The student documents the chosen widths and shows that no overflow occurs.

**Note**

The emulator specification written in T1.1 (registers, supported protocol features, timing, generator parameters) is the *reference* for both the RTL and its testbench. Write it before the RTL, and update it if the design changes.

# 6. ECC-Protected Memory and Register Interface

### 6.1 The (39,32) SECDED Hamming code

Each 32-bit data word $d$ is stored as a 39-bit code word: 6 Hamming check bits placed at positions 1, 2, 4, 8, 16 and 32 of a 38-bit Hamming word, plus one **overall parity** bit over all bits [A1, B1]. On read the decoder computes the 6-bit syndrome $s$ and the overall parity check $p$:

**Table 6.1:** SECDED decoding decisions.

| Syndrome $s$ | Overall parity | Decision |
|---|---|---|
| 0 | correct | no error |
| $\neq 0$ | wrong | single error at position $s \rightarrow$ **correct it** |
| 0 | wrong | error in the overall parity bit only (data is correct) |
| $\neq 0$ | correct | **double error** $\rightarrow$ detect and flag, do not correct |

Encoder and decoder are purely combinational modules (the student may add a pipeline register if timing requires it). A Hsiao code [A2] is a known alternative with a cheaper decoder; comparing the two is an

extension.

## 6.2 Sample record format (proposal)

Every measurement is stored as a record of three 32-bit words:

**Table 6.2:** Proposed sample record (each word is ECC-encoded separately).

| Word | Bits | Content |
|---|---|---|
| 0 header | [31:28] [27:24] [23:0] | sensor id, channel (X/Y/Z), sequence number |
| 1 timestamp | [31:0] | free-running microsecond counter |
| 2 value | [31:0] | raw sensor value, sign-extended |

## 6.3 Memory organisation and fault injection

- BRAM ring buffer, 1024 × 39 bits (inferred from SystemVerilog, not an IP core), with write pointer, wrap-around and an overflow flag.
- Write path: `data` → encoder → (fault-injection XOR mask) → BRAM.
- Read path: BRAM → decoder → corrected data, syndrome, single/double flags → error counters.
- Fault injection: the PS writes an address and a 39-bit mask; the next write to that address (or an immediate read-modify-write of the stored word) is XORed with the mask. 1-bit masks must be corrected, 2-bit masks must be detected.

## 6.4 AXI4-Lite register map (proposal)

**Table 6.3:** Proposed AXI4-Lite register map (32-bit registers).

| Offset | Name | Function |
|---|---|---|
| 0x00 | CONTROL | enable acquisition, clear buffer, clear counters |
| 0x04 | STATUS | busy, overflow, read-valid, last-read single/double error |
| 0x08 | SAMPLE_PERIOD | polling period in µs |
| 0x0C | SENSOR_EN | enable temperature / accelerometer / SYSMON |
| 0x10 | WR_PTR | current write pointer (read-only) |
| 0x14 | RD_ADDR | address to read; writing starts a decoded read |
| 0x18 | RD_DATA | corrected data of the last read |
| 0x1C | RD_SYNDROME | syndrome and error flags of the last read |
| 0x20 | ERR_CORR_CNT | number of corrected single-bit errors |
| 0x24 | ERR_DET_CNT | number of detected double-bit errors |
| 0x28 | INJ_ADDR | fault-injection address |
| 0x2C / 0x30 | INJ_MASK_LO / HI | 39-bit injection mask |
| 0x34 | INJ_CTRL | arm injection on next write / flip stored word now |
| 0x40–0x5C | GEN_* | value-generator settings of the emulators |
| 0xFC | VERSION | design version |

# 7. Processing System and Software

## 7.1 Hardware integration

The PL design is packaged as a custom IP with an AXI4-Lite slave interface and connected in a Vivado block design to the Zynq UltraScale+ PS (master port, PL clock and reset from the PS) [U4]. Address assignment, constraints, synthesis, implementation, timing analysis [U5] and bitstream generation complete the hardware platform, which is exported to Vitis.

## 7.2 Vitis application (MVP)

A bare-metal C application [U6] on one Cortex-A53 core:

1. configures the value generators and the sampling period, and starts acquisition;
2. reads new records from the ring buffer (`WR_PTR`, `RD_ADDR`/`RD_DATA`), decodes them into sensor, time and physical value (°C, $g$);
3. prints them over UART in CSV format, together with the error counters;
4. provides a small UART command menu to inject single and double errors at chosen addresses and to show the result.

## 7.3 Toward the server and web platform (extension)

Once the MVP works, the same records can be sent over Ethernet (e.g. the lwIP stack in Vitis) to a server program that stores them in a database and shows them on a web page (live plot per sensor, error counters). The CSV format of the MVP should be designed so that it can later be sent unchanged to the server.

# 8. Verification Plan

Verification is the main part of this project. It follows the usual industrial approach: **plan first, then verify bottom-up**: each module on its own, then the complete design. Every testbench is **self-checking**: it decides PASS or FAIL by itself, without anyone looking at waveforms [B2, B3].

## 8.1 Verification levels

**Table 8.1:** Verification levels, methods and pass criteria.

| Level / DUT | Stimulus and checking | Done when |
|---|---|---|
| ECC encoder + decoder | Random data words; for each, *all* 39 single-bit flips and all 741 double-bit flips; compare with a SystemVerilog reference model of the code. | 0 mismatches; every syndrome value and every error class covered. |
| TMP102 emulator | $I^2C$ master bus functional model (BFM) in the testbench (class-based); tests for address match/NACK, pointer write, register read/write, repeated START, both bus speeds; SVA on the protocol. | All register/operation combinations covered; 0 assertion failures. |
| ADXL345 emulator | SPI master BFM; single- and multi-byte reads/writes, DEVID, data registers, CS framing; SVA on mode-3 timing. | As above. |
| $I^2C$ / SPI masters | Run against the (already verified) emulators and the BFMs; random transaction sequences. | All transactions complete and match. |
| **Top level** | Class-based environment (Figure 8.1): constrained-random register traffic, sensor configurations and sampling periods; end-to-end scoreboard; fault-injection tests; buffer wrap and overflow tests. | Coverage goals of Section 8.3 met; 0 scoreboard errors over the regression. |
| On the board | Integrated Logic Analyzer (ILA) [U7] probes on the sensor buses and the memory interface; injections triggered from the Vitis application. | Corrected/detected counts match the injected errors. |

## 8.2 Top-level verification environment

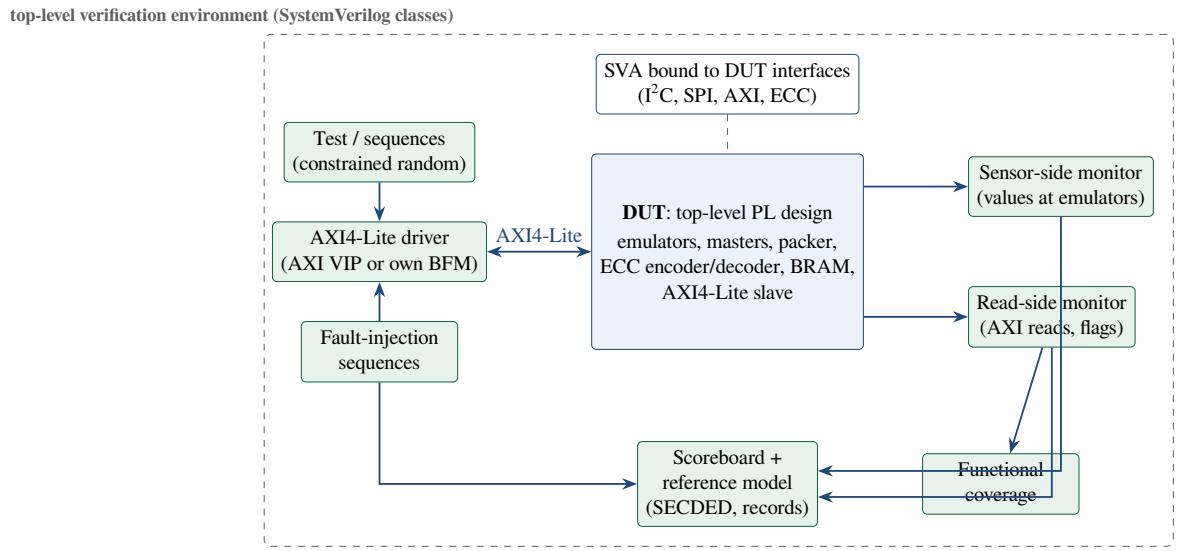



**Figure 8.1:** Structure of the top-level verification environment.

The environment is written with SystemVerilog classes and interfaces [B2]:

- **Driver**: drives AXI4-Lite transactions. The AMD AXI Verification IP [U8] can be used as master, or the student writes a small BFM.
- **Sequences / tests**: constrained-random configurations (enabled sensors, sampling period, generator settings), register access patterns and fault-injection scenarios, each with a seed for reproducibility.

- **Monitors**: one observes the values produced inside the emulators (what was *measured*), the other observes what the PS side *reads back* (data, syndrome, flags).
- **Scoreboard and reference model**: check end to end that every record read back equals the measured value (after correction), that the record format and sequence numbers are right, and that single errors are corrected and double errors flagged exactly where they were injected.
- **Assertions (SVA)** [B4]: protocol rules (e.g. SDA stable while SCL is high except at START/STOP; SPI data stable on the sampling edge; AXI handshakes), plus design rules (no write when the buffer is disabled; error flags never both set; counters never decrease except on clear).
- **Functional coverage**: covergroups and crosses for the items in Section 8.3.

A UVM version of this environment [B5] is optional (Vivado Simulator supports UVM); the required version uses plain SystemVerilog classes.

### 8.3 Coverage goals

- ECC: every single-bit error position (39), double errors in data, check and parity bits, no-error case; all decoder decisions of Table 6.1.
- Sensors: every register of both emulators read and written (where writable); all generator modes; anomaly events.
- Memory: empty, partly full, full, wrap-around, overflow; reads of the oldest and newest record; back-to-back reads.
- Configuration: all sensor-enable combinations $\times$ at least three sampling-period ranges.
- Target: **100%** of the coverage plan, or every uncovered item explained in the report.

### 8.4 Regression and reporting

All tests run from one script (the Vivado Simulator in batch mode, started from Tcl or a Makefile) with several random seeds [U9]. The script prints a PASS/FAIL summary and produces a coverage report. The regression must pass before every milestone review.

## 9. Tasks

**Individual topic:** *Design and verification of an ECC-protected sensor data memory in SystemVerilog on the AMD AUP ZU3.*

## 9.1 Specification

**T1.1** Confirm the sensor choice (Chapter 5) and write a one-page emulator specification for each sensor from its datasheet.

**T1.2** Write the verification plan: features to verify, test list, assertions and coverage items (refine Chapter 8).

**T1.3** Fix the record format and the AXI4-Lite register map.

## 9.2 Design (SystemVerilog RTL)

**T2.1** Implement the TMP102-style emulator ($I^2C$ target) with its value generator.

**T2.2** Implement the ADXL345-style emulator (SPI target) with its value generator.

**T2.3** Implement the $I^2C$ master and the SPI master.

**T2.4** Implement the sensor poller and the sample packer.

**T2.5** Implement the (39,32) SECDED encoder and decoder.

**T2.6** Implement the BRAM ring buffer with error counters and fault injection.

**T2.7** Implement the AXI4-Lite slave and integrate the top level.

**T2.8** (optional) Add the SYSMON reader as a third sensor.

## 9.3 Verification

**T3.1** Unit testbench for the ECC encoder/decoder with a reference model (exhaustive single- and double-bit errors).

**T3.2** Unit testbenches with $I^2C$ / SPI BFMs and SVA for both emulators and both masters.

**T3.3** Top-level class-based environment: driver, sequences, monitors, scoreboard, SVA, coverage.

**T3.4** Fault-injection, wrap-around and overflow tests; regression script with coverage report.

## 9.4 Implementation and software

**T4.1** Package the design as an AXI IP, build the Zynq block design, constrain, synthesize and implement; close timing; report resource usage.

**T4.2** Write the Vitis application: configuration, readout, CSV over UART, error-injection menu.

**T4.3** Validate on the board with ILA and software-triggered fault injection.

**Key Point**

The diploma is mainly judged on **RTL quality** and, above all, on the **verification**: the plan, the testbench architecture, the assertions, the coverage achieved and the proof that the ECC works.

# 10. Learning Path

## 10.1 Prerequisites

The student is expected to already have:

- Digital logic: combinational and sequential circuits, FSMs, timing diagrams. *Self-check:* design a counter and a small FSM and draw their waveforms. *If weak:* [B6], ch. 2–3.
- An HDL (Verilog or SystemVerilog): modules, always blocks, simulation. *Self-check:* write and simulate a parameterised register. *If weak:* [B6], ch. 4.
- C programming (pointers, bit operations), for the Vitis application.
- Binary arithmetic and XOR/parity, for the ECC.

## 10.2 What to learn

Aware — knows what it is and when it is used; Understand — can explain it and read existing work; Apply — can use it independently to build.

**Table 10.1:** Knowledge map.

| Topic | Level | Where it is used in the project | Resources |
|---|---|---|---|
| **SystemVerilog design** | | | |
| Synthesizable SystemVerilog: `always_ff/comb`, packages, interfaces, parameters, enums, structs | Apply | all RTL modules | [B7] |
| FSM design and clean reset/clocking | Apply | bus masters, emulators, poller | [B6], ch. 3; [B7] |
| Inferred block RAM | Apply | ring buffer | [U10] |
| **Error-correcting codes** | | | |
| Hamming codes, syndrome decoding, SECDED | Apply | encoder/decoder | [B1], ch. 4; [A1] |
| Hsiao codes; built-in BRAM ECC | Aware | comparison, extension | [A2]; [U10] |

| Topic | Level | Where it is used in the project | Resources |
|---|---|---|---|
| **Buses and sensors** | | | |
| $I^2C$ protocol (START/STOP, ACK, repeated START, timing) | Apply | TMP102 emulator, master | [U11] |
| SPI modes, CPOL/CPHA, multi-byte transfers | Apply | ADXL345 emulator, master | [U2] |
| Reading a sensor datasheet (registers, data formats) | Understand | emulator specifications | [U1, U2] |
| AXI4-Lite handshakes | Apply | register interface | [U12] |
| **Verification** | | | |
| Classes, randomization, constraints, mailboxes, interfaces | Apply | testbenches | [B2], ch. 4–8 |
| Layered testbench, scoreboard, reference model | Apply | top-level environment | [B2]; [B3] |
| SystemVerilog Assertions | Apply | protocol and design checks | [B4]; [B2] |
| Functional coverage (covergroups, crosses) | Apply | coverage plan | [B2], ch. 9 |
| UVM concepts | Aware | optional UVM environment | [B5] |
| AXI Verification IP | Understand | AXI driver | [U8] |
| **Zynq platform and tools** | | | |
| Zynq UltraScale+ PS/PL, AXI ports, clocks | Understand | block design | [U4]; [B8] |
| Vivado Simulator, batch mode and coverage | Apply | regression | [U9] |
| Timing analysis and closure | Understand | implementation | [U5] |
| Vitis bare-metal application, UART output | Apply | readout software | [U6] |
| ILA debugging | Understand | board validation | [U7] |

## 10.3 Tools and skills

- **Vivado** (Vivado Simulator, synthesis, implementation, ILA) and **Vitis**: free edition, same version on all machines **[ fix the tool version ]**.
- **git**: all code, scripts and documents in a repository from the first week.
- **Scripting** (Tcl or Python): regression and build scripts.
- The **IEEE 1800** SystemVerilog standard [U13] as reference when a book is not enough.

# 11. Getting Started

## 11.1 First meeting checklist

- Repository created; supervisor has access; folder structure agreed (`rtl/`, `tb/`, `sw/`, `scripts/`, `doc/`).
- AUP ZU3 board, power supply and cables handed over; lab access **[ lab / room ]**.
- Weekly meeting slot and communication channel.

## 11.2 Environment setup

1. Install Vivado and Vitis (agreed version) and the AUP ZU3 board files.
2. Build a minimal Zynq block design, generate the bitstream and run a "hello world" Vitis application that prints over UART.
3. Run a small SystemVerilog simulation from a script in batch mode (no GUI).

## 11.3 Warm-up exercises

Five short exercises, from simple to project-like. Each one practises a skill needed later, and each is kept in the repository with its own testbench.

**Warm-up exercise 1: Finite-state machine** — Est. time: 6 h

Design a Moore FSM that detects the bit pattern `1011` in a serial input (overlapping patterns allowed), then rewrite it as a Mealy FSM. Use `typedef enum` for the states and `always_ff`/`always_comb`. Write a testbench that sends random bit streams and checks the detector against a simple model (a shift register compared with `4'b1011`).

**Done when:** both versions pass 10 000 random bits; the student can draw both state diagrams and explain the one-cycle difference.

**Warm-up exercise 2: Memories: RAM and FIFO** — Est. time: 8 h

(a) Write a parameterised single-port and a simple dual-port RAM so that Vivado infers block RAM (check the synthesis report). (b) Build a synchronous FIFO on top of the dual-port RAM. Write a class-based testbench with random push/pop traffic, a queue-based reference model, two SVA properties (no write when full, no read when empty) and one covergroup (full, empty, simultaneous push and pop).

**Done when:** the synthesis report shows BRAM, not LUTs; the FIFO testbench reports PASS for 3 seeds and the coverage shows full and empty were reached.

**Warm-up exercise 3: SPI loop-back with assertions** Est. time: 8 h

Write a small SPI master (mode 3, 8-bit transfers) and a matching SPI target with one read/write register. Connect them in a testbench, write and read the register with random values, and add SVA: MOSI/MISO stable on the sampling edge, exactly 8 SCLK edges per byte while CS is low, MISO undriven when CS is high.

**Done when:** random write/read-back passes and all assertions are active (checked with a deliberately broken version that fails).

**Warm-up exercise 4: Small Hamming code** Est. time: 6 h

Implement a Hamming (7,4) encoder/decoder, extend it with an overall parity bit to SECDED (8,4), and test all 16 data words with all single and double errors.

**Done when:** the exhaustive test passes; the student can explain the syndrome table on paper.

**Warm-up exercise 5: First AXI4-Lite peripheral on the board** Est. time: 10 h

Create a custom AXI4-Lite IP with two registers (one written by the PS, one returning its value $+1$), connect it to the PS, and read/write it from a Vitis application.

**Done when:** the values printed over UART match, and the IP's own testbench passes.

### 11.4 Read first

1. [B2], chapters on classes, randomization and the layered testbench.
2. [B1], chapter on Hamming codes (or the original paper [A1]).
3. The TMP102 and ADXL345 datasheets [U1, U2]: protocol and register sections.

## 12. Work Plan and Milestones

The project runs over **30 weeks**. Weeks 14–15 fall on the winter holidays and the exam session. They are kept free as a buffer for catching up. The weekly checklist in Chapter 18 breaks this plan into tasks per week.

**Table 12.1:** Work plan (30 weeks).

| Weeks | Phase / milestone | Checkpoint result |
|---|---|---|
| 1–4 | Onboarding: tools, reading, five warm-up exercises (Chapter 11) | Warm-ups accepted |
| 5–6 | Specification: sensor choice, emulator specifications, record format, register map, verification plan | Specification and verification plan approved |

| Weeks | Phase / milestone | Checkpoint result |
|---|---|---|
| 7–9 | ECC encoder/decoder + unit testbench; ring buffer with error counters and fault injection | Exhaustive ECC test passes; code review 1 |
| 10–13 | Sensor emulators, $I^2C$/SPI masters + BFM testbenches + SVA | All unit testbenches pass; code review 2 |
| 14–15 | Buffer (holidays, exams): catch up; read on top-level testbenches | Nothing behind schedule |
| 16–19 | Top-level integration, AXI4-Lite slave; top-level environment, scoreboard, coverage | First end-to-end random test passes; code review 3 |
| 20–22 | Fault-injection and corner-case tests; regression; coverage closure | Coverage goals met; RTL and testbench frozen |
| 23–25 | Block design, implementation, timing closure; Vitis application; board validation | Demo on the board (Chapter 15) |
| 26–28 | Writing the diploma document | Full draft to the supervisor (week 28) |
| 29–30 | Revisions, slides, demo rehearsal | Final document and presentation ready |

**Note**

Weekly meeting with the supervisor **[ day / time ]**. Before each meeting the student sends a short note: what was done, what is blocked, next steps, and the latest regression result. The tracker in Chapter 18 is updated at every meeting.

# 13. Deliverables

**Table 13.1:** Project deliverables.

| Deliverable | Form | Due | Responsible |
|---|---|---|---|
| Emulator specifications, record format, register map | Markdown/PDF in the repository | week 6 | Student |
| Verification plan | document in the repository | week 6 | Student |
| SystemVerilog RTL | git repository, README | week 22 | Student |
| Testbenches, assertions, coverage model, regression script | git repository; regression and coverage reports | week 22 | Student |
| Vivado project / build script, Vitis application | git repository with build instructions | week 25 | Student |
| Implementation results | timing and utilization reports | week 25 | Student |
| Diploma document | PDF (diploma-project template) | draft week 28, final week 30 | Student |

| Deliverable | Form | Due | Responsible |
|---|---|---|---|
| Warm-up exercises (code + testbenches) | git repository | week 4 | Student |
| Defense presentation and demo | slides + live demo or video | week 30 | Student |

# 14. Assessment and Acceptance Criteria

## 14.1 Acceptance criteria

The project is accepted for the defense when:

- the MVP (Chapter 4) works on the AUP ZU3 and is demonstrated;
- all unit testbenches and the top-level regression pass;
- all injected single-bit errors are corrected and all injected double-bit errors are detected, in simulation and on the board;
- the design meets timing;
- the code, scripts and instructions are in the repository, and the build and regression can be repeated by another person.

## 14.2 Assessment rubric

**Table 14.1:** Assessment rubric.

| Criterion | Weight | Minimum (pass) | Good | Excellent |
|---|---|---|---|---|
| RTL design | 25% | MVP modules work | Clean, parameterised, well-structured RTL; documented interfaces | + optional SYSMON sensor or design-space comparison (e.g. Hsiao vs. Hamming) |
| Verification | 30% | Unit testbenches self-checking; top level passes directed tests | Constrained-random top-level env with scoreboard, SVA and coverage goals met | + coverage closure fully explained, UVM version, or bugs found and documented |
| Hardware integration | 15% | Runs on board, data read over UART | Timing met with margin, ILA validation, fault injection from software | + extension (server/DB/web) |
| Understanding and analysis | 10% | Explains ECC and the protocols | Analyses resources, timing and test results | Compares alternatives with numbers |

| Criterion | Weight | Minimum (pass) | Good | Excellent |
|---|---|---|---|---|
| Work process | 5% | Milestones mostly met | Regular commits, weekly notes, tracker kept up to date | Independent, anticipates problems |
| Document and defense | 15% | Complete document | Clear structure, figures, verification results | Publication-quality text and demo |

**Note**

The rubric supplements, and does not replace, the official grading rules of the university.

# 15. Final Demonstration

1. Show the regression in the terminal: all tests PASS; open the coverage report.
2. Program the AUP ZU3; the Vitis application starts acquisition with both sensors.
3. Temperature and acceleration records appear over UART as CSV; the student explains one record (header, timestamp, value).
4. Trigger an "overheating" anomaly in the temperature emulator: the change appears in the data.
5. Inject a **single-bit** error at a chosen address: the record is read back correctly, and the corrected-error counter increases.
6. Inject a **double-bit** error: the read is flagged as uncorrectable, and the detected-error counter increases.
7. Show the ILA capture of an $I^2C$ and an SPI transaction.

# 16. Extensions (Optional)

The following items are **not required**. They can raise the grade or become a follow-up project.

- **Server, database and web platform:** send the records over Ethernet (lwIP) to a server program, store them in a database (e.g. SQLite or PostgreSQL), and show live plots and error counters on a web page. This completes the original vision of the system.
- **Real sensors:** connect a TMP102 and an ADXL345 breakout board to the board's expansion connectors and use the same $I^2C$/SPI masters. The emulators become the reference models.
- **Memory scrubbing:** a background process reads every word, corrects single errors and writes the corrected word back.
- **Code comparison:** Hsiao vs. Hamming SECDED, or own ECC vs. the built-in BRAM ECC [U10]: area, timing, latency.
- **UVM environment** for the top level [B5].

- **Third sensor:** SYSMON die temperature and voltages [U3].

# 17. Supervision, Collaboration and Risks

## 17.1 Supervision

- **Supervisor:** Arman Manukyan, PhD. Weekly meeting; reviews the specification, the verification plan and every milestone.
- **Code reviews:** the supervisor reviews the RTL and testbench structure at weeks 9, 13 and 19.
- **Collaboration:** git repository; questions between meetings via **[ channel ]**.

## 17.2 Risks

**Table 17.1:** Main risks and mitigation.

| Risk | Impact | Mitigation |
|---|---|---|
| Learning curve of class-based verification | high | Warm-ups 2–3, [B2] early, start with a simple environment and grow it |
| $I^2C$ open-drain behaviour inside the FPGA | medium | Wired-AND modelling (Chapter 4); SVA on the bus |
| Scope too large for one student | medium | Server/web is an extension only; SYSMON is optional; buffer weeks 14–15 |
| Timing problems at the target clock | low | Register ECC outputs; lower PL clock for the MVP |
| Board not available | low | Everything except the final validation runs in simulation |

# 18. Weekly Progress Tracker

Tick the box when the week's planned work is done, and write a short note (or the supervisor's initials) in the last column at the weekly meeting. Task numbers refer to Chapter 9; warm-ups to Chapter 11. Two unticked weeks in a row means the plan must be discussed and adjusted.

**Table 18.1:** Weekly progress tracker.

| Week | Planned work (tasks) | Checkpoint result | ✓ | Notes / initials |
|---|---|---|---|---|
| **Onboarding** | | | | |
| 1 | Install Vivado/Vitis, board files, repository; "hello world" on the board. Warm-up 1 (FSM) | Tools run; Warm-up 1 passes | ☐ | |
| 2 | Warm-up 2 (RAM and FIFO); read [B2] on classes and randomization | Warm-up 2 passes | ☐ | |
| 3 | Warm-up 3 (SPI loop-back + SVA); Warm-up 4 (Hamming code) | Warm-ups 3–4 pass | ☐ | |
| 4 | Warm-up 5 (AXI4-Lite IP on the board); read TMP102 / ADXL345 datasheets | All warm-ups accepted | ☐ | |
| **Specification** | | | | |
| 5 | T1.1 sensor choice and emulator specifications | Specs reviewed | ☐ | |
| 6 | T1.2 verification plan; T1.3 record format and register map | Specification approved | ☐ | |
| **ECC memory** | | | | |
| 7 | T2.5 SECDED encoder/decoder; start T3.1 reference model | Encoder/decoder simulate | ☐ | |
| 8 | T3.1 exhaustive ECC testbench; start T2.6 ring buffer | 0 mismatches over all 1- and 2-bit errors | ☐ | |
| 9 | T2.6 error counters and fault injection + unit tests | ECC memory verified; code review 1 | ☐ | |
| **Sensors and buses** | | | | |
| 10 | T2.1 TMP102 emulator; $I^2C$ BFM | Emulator answers address/pointer reads | ☐ | |

| Week | Planned work (tasks) | Checkpoint result | ✓ | Notes / initials |
|---|---|---|---|---|
| 11 | T2.3 $I^2C$ master; T3.2 $I^2C$ tests + SVA | $I^2C$ testbenches pass | ☐ | |
| 12 | T2.2 ADXL345 emulator; SPI BFM; T2.3 SPI master | DEVID and burst read work | ☐ | |
| 13 | T3.2 SPI tests + SVA; coverage of both emulators | All unit testbenches pass; code review 2 | ☐ | |
| **Buffer** | | | | |
| 14 | Catch up on open items; read [B2] on the complete testbench | Nothing behind schedule | ☐ | |
| 15 | Catch up; sketch the top-level environment (Figure 8.1) | Environment structure agreed | ☐ | |
| **Integration and top-level verification** | | | | |
| 16 | T2.4 sensor poller and sample packer | Records produced in simulation | ☐ | |
| 17 | T2.7 AXI4-Lite slave; top-level integration | Top level simulates end to end | ☐ | |
| 18 | T3.3 driver, sequences, monitors | Random register traffic runs | ☐ | |
| 19 | T3.3 scoreboard, SVA, coverage | First end-to-end random test passes; code review 3 | ☐ | |
| 20 | T3.4 fault-injection tests | All injected errors corrected / detected | ☐ | |
| 21 | T3.4 wrap-around and overflow tests; regression script | Regression runs with several seeds | ☐ | |
| 22 | Coverage closure; fix bugs; freeze RTL and testbench | Coverage goals met | ☐ | |
| **Hardware and software** | | | | |
| 23 | T4.1 block design, constraints, implementation, timing | Bitstream with timing met | ☐ | |
| 24 | T4.2 Vitis application: readout, CSV over UART, injection menu | Data visible over UART | ☐ | |
| 25 | T4.3 board validation with ILA; resource and timing reports | Demo on the board works | ☐ | |
| **Document and defense** | | | | |

| Week | Planned work (tasks) | Checkpoint result | ✓ | Notes / initials |
|---|---|---|---|---|
| 26 | Write introduction, background, design chapters | Chapters to supervisor | ☐ | |
| 27 | Write verification and results chapters | Chapters to supervisor | ☐ | |
| 28 | Complete draft: conclusion, abstract, references | Full draft submitted | ☐ | |
| 29 | Revisions after feedback; defense slides | Slides draft | ☐ | |
| 30 | Final document; demo rehearsal | Ready for the defense | ☐ | |

**Note**

Extensions (Chapter 16) and the optional task T2.8 are started only when the tracker is up to date.

# References and Reading List

## Books


[B1] Shu Lin and Daniel J. Costello. *Error Control Coding*. 2nd ed. Pearson Prentice Hall, 2004.

[B2] Chris Spear and Greg Tumbush. *SystemVerilog for Verification: A Guide to Learning the Testbench Language Features*. 3rd ed. Springer, 2012.

[B3] Janick Bergeron, Eduard Cerny, Alan Hunter, and Andy Nightingale. *Verification Methodology Manual for SystemVerilog*. Springer, 2006.

[B4] Eduard Cerny, Surrendra Dudani, John Havlicek, and Dmitry Korchemny. *SVA: The Power of Assertions in SystemVerilog*. 2nd ed. Springer, 2015.

[B5] Ray Salemi. *The UVM Primer: A Step-by-Step Introduction to the Universal Verification Methodology*. Boston Light Press, 2013.

[B6] Sarah L. Harris and David Money Harris. *Digital Design and Computer Architecture: ARM Edition*. Morgan Kaufmann, 2016.

[B7] Stuart Sutherland, Simon Davidmann, and Peter Flake. *SystemVerilog for Design: A Guide to Using SystemVerilog for Hardware Design and Modeling*. 2nd ed. Springer, 2006.

[B8] Louise H. Crockett, Ross A. Elliot, Martin A. Enderwitz, and Robert W. Stewart. *The Zynq Book: Embedded Processing with the ARM Cortex-A9 on the Xilinx Zynq-7000 All Programmable SoC*. Strathclyde Academic Media, 2014.

### Articles and papers

[A1] Richard W. Hamming. “Error Detecting and Error Correcting Codes”. In: *Bell System Technical Journal* 29.2 (1950), pp. 147–160.

[A2] M. Y. Hsiao. “A Class of Optimal Minimum Odd-weight-column SEC-DED Codes”. In: *IBM Journal of Research and Development* 14.4 (1970), pp. 395–401.

### User guides, datasheets and standards

[U1] Texas Instruments. *TMP102 Low-Power Digital Temperature Sensor With SMBus and Two-Wire Serial Interface (datasheet)*. Texas Instruments.

[U2] Analog Devices. *ADXL345 3-Axis, ±2 g/±4 g/±8 g/±16 g Digital Accelerometer (datasheet)*. Analog Devices.

[U3] AMD. *UltraScale Architecture System Monitor User Guide (UG580)*. Advanced Micro Devices.

[U4] AMD. *Zynq UltraScale+ Device Technical Reference Manual (UG1085)*. Advanced Micro Devices.

[U5] AMD. *Vivado Design Suite User Guide: Design Analysis and Closure Techniques (UG906)*. Advanced Micro Devices.

[U6] AMD. *Vitis Unified Software Platform Documentation: Embedded Software Development (UG1400)*. Advanced Micro Devices.

[U7] AMD. *Integrated Logic Analyzer LogiCORE IP Product Guide (PG172)*. Advanced Micro Devices.

[U8] AMD. *AXI Verification IP LogiCORE IP Product Guide (PG267)*. Advanced Micro Devices.

[U9] AMD. *Vivado Design Suite User Guide: Logic Simulation (UG900)*. Advanced Micro Devices.

[U10] AMD. *UltraScale Architecture Memory Resources User Guide (UG573)*. Advanced Micro Devices.

[U11] NXP Semiconductors. *I2C-bus Specification and User Manual (UM10204)*. NXP Semiconductors.

[U12] Arm. *AMBA AXI Protocol Specification (IHI 0022)*. Arm Limited.

[U13] IEEE. *IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language (IEEE Std 1800-2017)*. IEEE. 2018.

# List of Abbreviations

**ECC** Error-Correcting Code.

**SECDED** Single-Error Correction, Double-Error Detection.

**FPGA** Field-Programmable Gate Array.

**PL** Programmable Logic.

**PS** Processing System.

**MPSoC** Multiprocessor System-on-Chip.

**RTL** Register-Transfer Level.

**BRAM** Block RAM.

**AXI** Advanced eXtensible Interface.

**$I^2C$** Inter-Integrated Circuit bus.

**SPI** Serial Peripheral Interface.

**SYSMON** System Monitor (on-chip temperature and voltage sensor).

**LFSR** Linear-Feedback Shift Register.

**FSM** Finite-State Machine.

**SVA** SystemVerilog Assertions.

**UVM** Universal Verification Methodology.

**BFM** Bus Functional Model.

**DUT** Device Under Test.

**VIP** Verification IP.

**ILA** Integrated Logic Analyzer.

**UART** Universal Asynchronous Receiver-Transmitter.

**MVP** Minimum Viable Product.

# Glossary

**syndrome** the check bits recomputed on read XOR the stored check bits; zero means no error, otherwise it points to the flipped bit.

**fault injection** deliberately flipping bits in stored words to prove that the ECC corrects and detects errors.

**scoreboard** testbench component that compares the DUT output with the output of a reference model.

**functional coverage** measurement of which scenarios, values and corner cases the tests have actually exercised.

**sensor emulator** RTL module that behaves like a real sensor chip on its bus (registers, protocol, timing) and produces synthetic measurement values.